

Nuvem e Sistemas Operacionais

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br

Sumário

- Fundamentos da Computação em Nuvem
- Características, Virtualização e Recursos em Cloud
- Modelos de Serviço e Implantação
- Provedores, Vantagens, Desafios e Segurança
- Containers, Microsserviços e Backend
- API REST com Express e Deploy no Render
- Atividade

Conceito de Cloud Computing

Acesso sob Demanda

Recursos alocados instantaneamente, sem intervenção humana manual.

Rede de Acesso Amplo

Acesso via dispositivos variados: PCs, smartphones e tablets.

Pool de Recursos

Recursos compartilhados e alocados dinamicamente entre múltiplos usuários.

Mensuração Automática

Uso monitorado e medido automaticamente para cobrança e controle.

Características da Computação em Nuvem

As principais características (modelo NIST) são:

- Autoatendimento sob demanda: Recursos podem ser provisionados automaticamente pelo usuário.
- Amplo acesso à rede: Serviços acessíveis via internet por diferentes dispositivos.
- Pool de recursos: Recursos compartilhados entre múltiplos usuários (multi-tenant).
- Elasticidade: Capacidade de escalar recursos rapidamente conforme a demanda.
- Serviço mensurável: Uso monitorado e cobrado conforme consumo (pay-per-use).

Infraestrutura Tradicional para Cloud

A evolução para a nuvem representou uma mudança de paradigma: de ativos físicos próprios para serviços contratados sob demanda. As organizações deixaram de investir em hardware (CAPEX) e passaram a pagar apenas pelo que consomem (OPEX), ganhando agilidade e reduzindo custos operacionais.



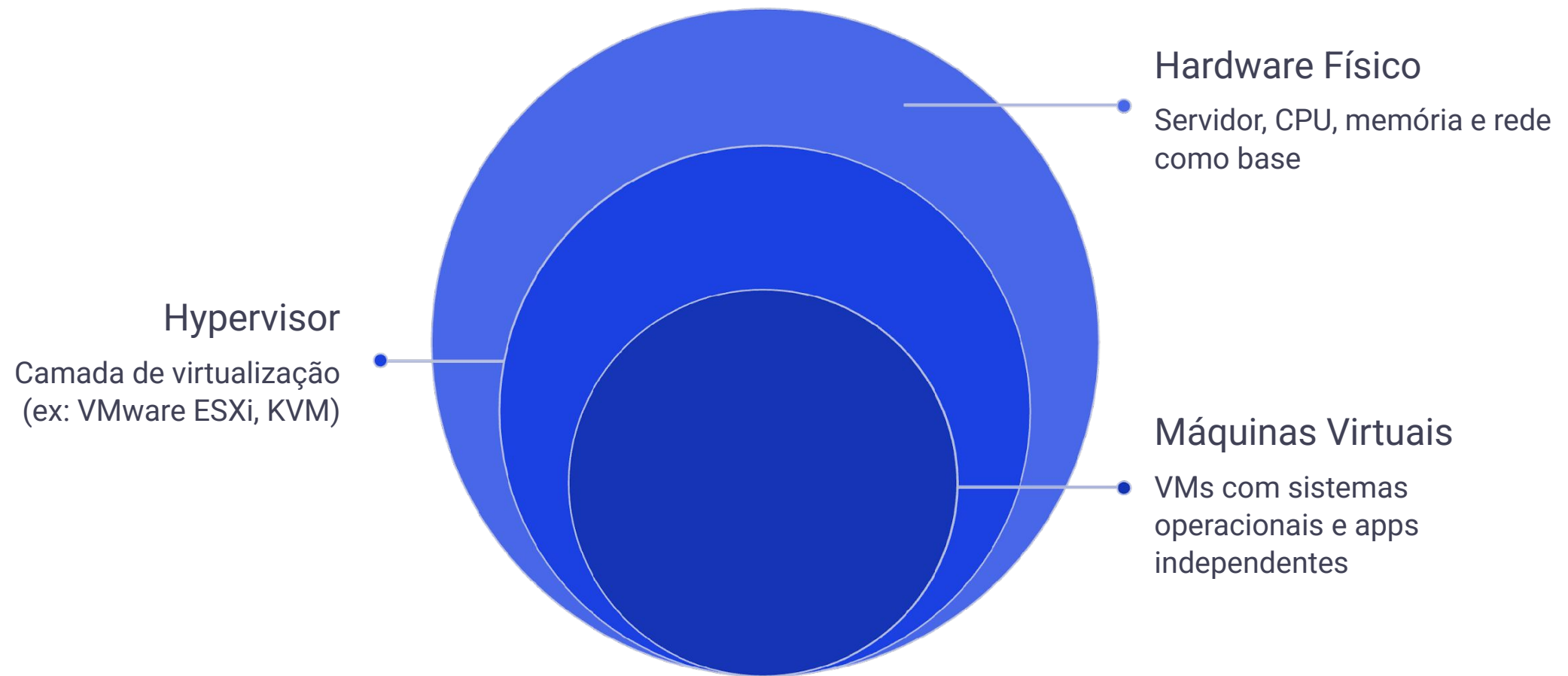
Fonte: tecwi.com.br

Virtualização como Base da Nuvem

O Sistema Operacional é a camada fundamental que abstrai o hardware e gerencia recursos.

A virtualização permite criar múltiplas máquinas virtuais (VMs) sobre um único servidor físico, isolando ambientes e otimizando recursos. O hypervisor (ou VMM) é o software responsável por essa abstração.

Virtualização como Base da Nuvem



Recursos Computacionais sob Demanda

Um dos princípios fundamentais da nuvem é o self-service sob demanda: o usuário provisiona recursos sem interação humana com o provedor.

Elasticidade é a capacidade de expandir ou contrair recursos automaticamente conforme a demanda. A escalabilidade refere-se à capacidade do sistema de crescer para atender cargas maiores.

Alta Disponibilidade em Cloud

1

Zonas de Disponibilidade

Datacenters fisicamente separados dentro de uma mesma região, com energia e rede independentes.

2

Balanceamento de Carga

Distribui requisições entre múltiplas instâncias, evitando sobrecarga e ponto único de falha.

3

Replicação de Dados

Dados replicados sincronamente entre zonas para garantir continuidade em caso de falha.

4

Failover Automático

Em caso de falha, o tráfego é redirecionado automaticamente para instâncias saudáveis.

Modelos de Serviço: IaaS

- O provedor disponibiliza **infraestrutura** virtualizada: servidores, armazenamento e rede sob demanda
- O cliente gerencia SO, middleware, runtime e aplicações, maior controle e flexibilidade
- Escala elástica: recursos alocados ou liberados conforme a necessidade, com pagamento por uso
- Exemplos: AWS EC2, Google Compute Engine, Azure Virtual Machines
- Ideal para migração de datacenters físicos e ambientes que exigem controle do SO

Modelos de Serviço: PaaS

- O provedor gerencia toda a infraestrutura e a **plataforma** de execução: SO, runtime e middleware
- O desenvolvedor foca exclusivamente no código e na lógica da aplicação
- Reduz complexidade operacional e acelera o ciclo de desenvolvimento e deploy
- Exemplos: Google App Engine, Azure App Service, Heroku, AWS Elastic Beanstalk
- Indicado para equipes ágeis que desejam reduzir overhead de gerenciamento de infraestrutura

Modelos de Serviço: SaaS

- **Software** completo entregue via internet — o usuário acessa sem instalar ou gerenciar nada
- Toda a infraestrutura, plataforma e software são gerenciados pelo provedor
- Modelo de assinatura (subscription) com atualizações automáticas e alta disponibilidade
- Exemplos: Google Workspace, Microsoft 365, Salesforce, Slack, Zoom
- Reduz custos de licenciamento e elimina a necessidade de manutenção local de software

Principais Provedores de Cloud



AWS — Amazon Web Services

Líder global com mais de 200 serviços. Maior quota de mercado e ecossistema mais maduro.



Azure — Microsoft

Forte integração com ecossistema Microsoft.
Dominante em ambientes corporativos híbridos.



GCP — Google Cloud Platform

Destaque em Big Data, IA/ML e containers (Kubernetes foi criado pelo Google).



Outros Provedores

Oracle Cloud, IBM Cloud, Alibaba Cloud e provedores regionais como Locaweb e AWS Local Zones no Brasil.

Modelos de Implantação em Cloud

Nuvem Pública

Infraestrutura compartilhada entre múltiplos clientes. Alta escalabilidade e menor custo. Ex: AWS, Azure, GCP.

Nuvem Privada

Infraestrutura dedicada a uma única organização. Maior controle, segurança e customização. Pode ser on-premise ou terceirizada.

Nuvem Híbrida

Combina nuvem pública e privada. Cargas sensíveis ficam on-premise; picos de demanda usam a nuvem pública.
Modelo mais adotado por grandes empresas.

Vantagens e Desafios da Cloud

✓ Vantagens

- Redução de CAPEX — substitui investimento em hardware por modelo operacional (OPEX)
- Escalabilidade elástica e sob demanda
- Alta disponibilidade e resiliência geográfica
- Acesso global e colaboração remota facilitada
- Inovação acelerada com serviços gerenciados de IA, analytics e IoT

⚠ Desafios

- Vendor lock-in: dependência de APIs e serviços proprietários
- Segurança e conformidade regulatória (LGPD, GDPR)
- Custos imprevisíveis sem governança adequada (FinOps)
- Latência de rede em aplicações sensíveis
- Gestão de ambientes multicloud e híbridos complexos

Segurança em Cloud Computing

- Modelo de Responsabilidade Compartilhada: O provedor garante a segurança da nuvem (infraestrutura física); o cliente garante a segurança na nuvem (dados, acesso, configurações).
 - Proteção de dados armazenados e em envio
 - Controle de acesso com login seguro
 - Monitoramento constante de atividades suspeitas
 - Adequação a normas e leis de proteção de dados

Containers e Microserviços

Containers

Empacotam aplicação + dependências em unidades isoladas, compartilhando o kernel do SO hospedeiro. Leves, portáteis e com inicialização rápida.

- Docker: padrão de fato para criação e distribuição de containers
- Kubernetes: orquestração, escalabilidade automática e alta disponibilidade

Microserviços

Arquitetura que divide aplicações em serviços independentes, cada um com responsabilidade única, comunicando-se via API.

- Facilita deploy independente, escalabilidade granular e resiliência
- Base tecnológica das aplicações cloud-native modernas

Servidor Backend e Web Service

- Servidor Backend: Sistema que processa requisições, gerencia dados e fornece respostas para clientes (apps, navegadores).
- Funções Principais:
 - Armazenar/recuperar dados (banco de dados)
 - Executar regras de negócio
 - Fornecer APIs para comunicação
- Web Service: Serviço acessível via web que permite comunicação entre sistemas usando HTTP/HTTPS.
 - Permite que sistemas heterogêneos (diferentes linguagens, plataformas ou tecnologias) se comuniquem de forma padronizada.

Criando uma API REST com Express

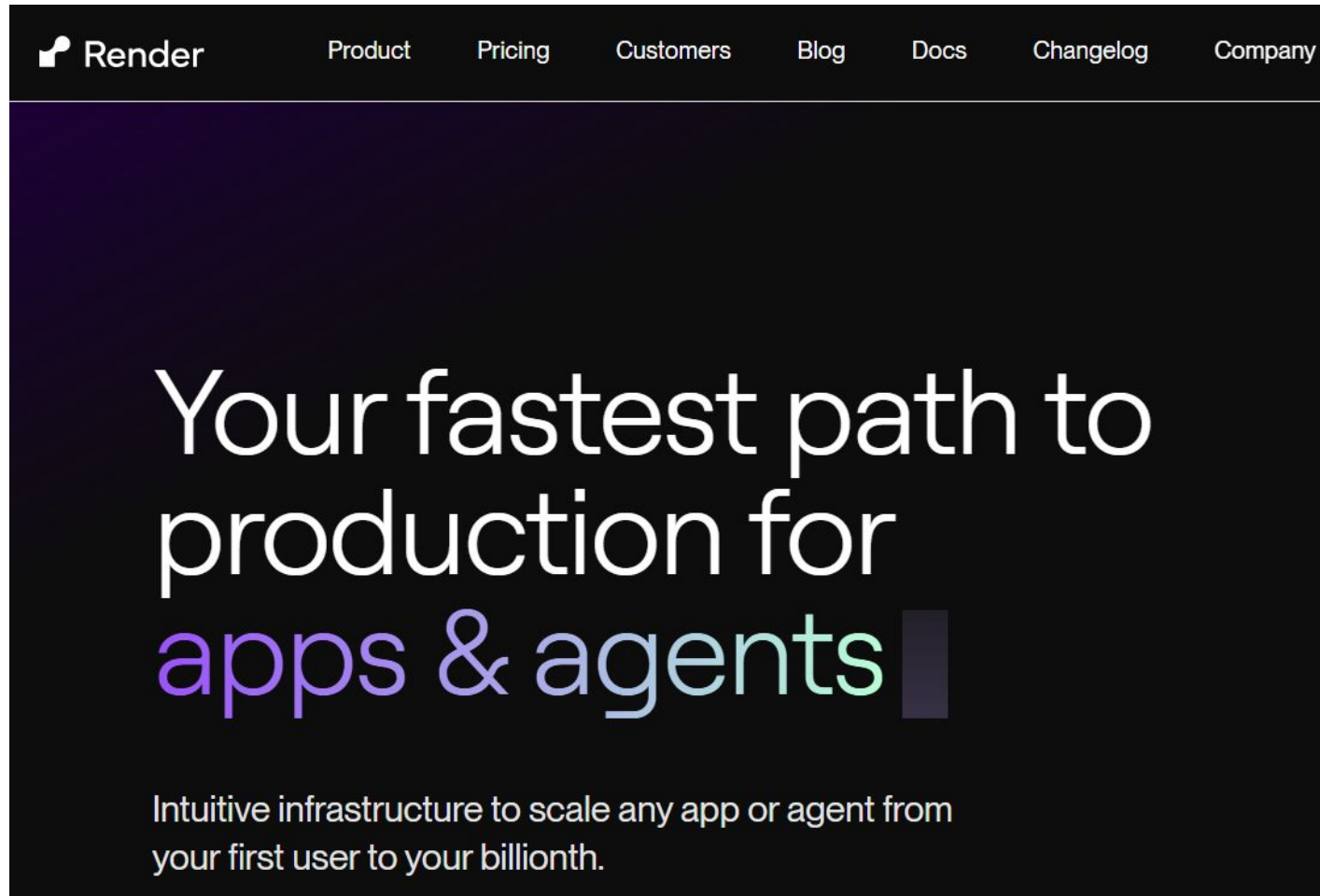
- # Passo 1: Inicializando o projeto
 - Crie uma nova pasta para seu projeto e abra no VS Code.
- # Passo 2: Instale o Express.js
 - `npm install express`
- # Passo 3: Instale o Cors
 - `npm install cors express`
- # Passo 4: Criando o arquivo api.js
 - Seguir o exemplo: <https://github.com/devisontakatu/exemplo-express-so/blob/main/index.js>
- # Passo 5: Executando o servidor
 - `node index.js`

CORS é um mecanismo de segurança que controla acesso entre domínios diferentes no navegador.

Render para Simular Web Services

- Plataforma de hospedagem em nuvem moderna. Suporta Node.js, Python e outras linguagens. Integração fácil com repositórios Git.
- Oferece deploy contínuo automático. Planos gratuitos para projetos pequenos. Interface simples e intuitiva para iniciantes.
- Escalável para aplicações profissionais. Certificado SSL incluso gratuitamente. Ideal para APIs e microsserviços.

Render para Simular Web Services



The screenshot shows the Render website homepage. At the top is a dark navigation bar with the Render logo and links for Product, Pricing, Customers, Blog, Docs, Changelog, and Company. The main content area has a dark background with the headline "Your fastest path to production for apps & agents" in white and colorful text. Below the headline is a sub-headline: "Intuitive infrastructure to scale any app or agent from your first user to your billionth."

Render

Product Pricing Customers Blog Docs Changelog Company

Your fastest path to
production for
apps & agents

Intuitive infrastructure to scale any app or agent from
your first user to your billionth.

Fonte: render.com

Benefícios do Render

- Deploy rápido em poucos cliques. Configuração simplificada sem terminal. Atualizações automáticas via GitHub.
- Ambiente de produção profissional. Escalabilidade automática sob demanda. Monitoramento de desempenho integrado.
- Suporte técnico eficiente. Infraestrutura confiável e segura. Perfeito para projetos acadêmicos.

Criando uma API REST com Express

- # Passo 1: Commit do Projeto no Github
 - Deixe disponível o projeto em um repositório do Github
- # Passo 2: Criando a conta no Render
 - Acesse dashboard.render.com
- # Passo 3: Crie novo "Web Service"
 - Clique no botão New e conecte o repositório no Github
- # Passo 4: Defina comando de start
 - Em Build Command: node / Em Start Command: node index.js
- # Passo 5: Deploy web service
 - Após realizar o deploy, utilize: seu-projeto.onrender.com

Atividade

1. Crie uma aplicação utilizando Express.js com o nome cloud-so-app. Desenvolva uma página web que exiba informações do sistema operacional utilizando recursos do Node.js, como nome do host, plataforma, arquitetura, quantidade de CPUs, memória total, memória livre e tempo de atividade do sistema. Teste a aplicação localmente e verifique seu funcionamento no navegador;

Atividade

2. Crie um repositório no GitHub e envie o projeto desenvolvido. Em seguida, crie uma conta no Render e publique a aplicação em ambiente de nuvem. Acesse o sistema online, compare as informações exibidas com a sua máquina local e analise as diferenças entre execução local e cloud;

Atividade

3. Identifique na aplicação os conceitos de Sistemas Operacionais estudados em aula, relacionando o projeto com processos, gerenciamento de memória, uso de CPU, sistema operacional hospedeiro, virtualização e computação em nuvem. Registre suas observações de forma técnica e organizada;

Atividade

4. Por fim, documente todo o processo na forma de Manual, contendo instalação das ferramentas, criação do projeto, desenvolvimento da aplicação, publicação no Render, testes realizados, comparações entre ambientes e conclusões finais. Salve o material no repositório da disciplina.

Referências

- TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.
- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.
- STALLINGS, William. Sistemas Operacionais: Conceitos e Projetos. 8. ed. São Paulo: Pearson, 2015.
- DENARDIN, G. W.; BARRIQUELLO, C. H. Sistemas Operacionais de Tempo Real e sua Aplicação em Sistemas Embarcados. Porto Alegre: Editora da UFRGS, 2014.
- AWASTHI, A.; RAWAT, V. Ramificação e Tarefas do Sistema Operacional. Edições Nosso Conhecimento, 2023.
- DOWNEY, Allen B. Think OS: A Brief Introduction to Operating Systems. Green Tea Press, 2015.
- RED HAT. Red Hat Enterprise Linux – System Administration Guide. Documentação Oficial.
- DOCKER INC. Docker Documentation. Documentação Oficial. Disponível em: <https://docs.docker.com>

Nuvem e Sistemas Operacionais

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br